

Atelier Machine Learning

Analyse Comportementale Clientèle Retail

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par Mohamed Ayoub Ben Ayed GI2 S1

Année Universitaire : 2025-2026

1 Prétraitement des Données (Preprocessing)

1.1 Objectif

Le prétraitement des données constitue une étape essentielle dans tout projet de Machine Learning. Son objectif est de transformer les données brutes en un format exploitable par les algorithmes d'apprentissage automatique, tout en améliorant la qualité et la pertinence des variables.

Dans ce projet, un pipeline complet de preprocessing a été développé afin de :

- Nettoyer les données brutes (identifiants, colonnes constantes)
- Générer de nouvelles variables pertinentes (feature engineering)
- Gérer les valeurs manquantes par imputation
- Encoder les variables catégorielles
- Normaliser les variables numériques
- Réduire la redondance par suppression des features corrélées et multicollinéaires
- Détecter et supprimer les features causant du *data leakage*

Vue d'ensemble du pipeline

Le pipeline est structuré en **trois phases** selon leur rapport au split train/test :

	Phase	Opérations
1	Avant le split (transformations déterministes, pas de fit sur les données)	Parsing des dates, featurisation IP, variables dérivées, suppression de CustomerID et des colonnes constantes
2	Après le split (fit sur train uniquement pour éviter le leakage)	Imputation, normalisation, encodage, suppression par corrélation, suppression par VIF
3	Détection du leakage (après encodage et VIF)	Mesure de la corrélation de chaque feature avec la cible $ r > 0.3$, suppression des variables proxies ou directement liées au target

1.2 Justification de l'ordre de traitement choisi

L'ordre des étapes du pipeline n'est pas arbitraire. Chaque décision repose sur un principe fondamental du Machine Learning : **ne jamais laisser le jeu de test influencer les décisions prises sur le jeu d'entraînement**. Le tableau suivant résume la logique de chaque choix :

#	Étape	Placée	Pourquoi ce choix ?
1	Feature engineering (dates, IP, ratios)	Avant le split	Transformations purement mathématiques et déterministes. Elles ne dépendent d'aucune statistique des données, donc elles ne peuvent pas causer de leakage, même appliquées sur tout le dataset.
2	Suppression de <code>CustomerID</code> et des colonnes constantes	Avant le split	Règles basées sur la structure des données (identifiant, variance nulle), pas sur des statistiques apprises. Aucun risque de leakage.
3	Split train / test	Pivot central	C'est le point de séparation critique. Tout ce qui suit doit être appris uniquement sur train. Placer le split ici garantit que le test reste totalement inconnu du pipeline.
4	Imputation, normalisation, encodage (sklearn)	Après le split	Ces étapes apprennent des statistiques sur les données (médiane, moyenne, catégories). Les apprendre sur tout le dataset ferait fuir de l'information du test vers le train (<i>data leakage</i>).
5	Suppression par corrélation	Après le split (après encodage)	La corrélation est calculée sur les features encodées/normalisées. Elle doit être mesurée sur <code>X_train</code> uniquement pour que la décision de supprimer une feature ne soit pas influencée par la distribution du test. De plus, elle doit être placée après l'encodage car <code>OneHotEncoder</code> crée de nouvelles colonnes qui peuvent elles-mêmes être corrélées.
6	Suppression par VIF	Après le split (après corrélation)	Le VIF mesure la multicolinéarité résiduelle après le filtre de corrélation. Il est placé en dernier car il est plus coûteux (régression itérative) et complémentaire : il détecte

Conséquence du non-respect de cet ordre

Si l'imputation, la normalisation ou la suppression par corrélation étaient effectuées **avant** le split :

- La médiane et l'écart-type seraient calculés sur train **et** test → le modèle aurait une connaissance indirecte du test
- La sélection de features serait biaisée par la distribution complète des données
- Les métriques d'évaluation (accuracy, AUC...) seraient **artificiellement gonflées** et ne refléteraient pas les performances réelles en production

Ce phénomène s'appelle le **data leakage** et constitue l'une des erreurs les plus fréquentes et les plus graves en Machine Learning.

1.3 Phase 1 — Transformations déterministes (avant le split)

Ces opérations sont appliquées sur le jeu de données complet **avant** la séparation train/test. Elles ne nécessitent aucun apprentissage statistique sur les données : elles n'induisent donc **aucun risque de data leakage**.

Feature Engineering

Transformation des dates

La variable `RegistrationDate` a été décomposée en :

- Année (`RegYear`), Mois (`RegMonth`), Jour (`RegDay`), Jour de la semaine (`RegWeekday`)

Cette transformation permet au modèle de capturer des comportements temporels saisonniers.

Transformation des adresses IP

La variable `LastLoginIP` a été transformée en :

- `IsPrivateIP` : indique si l'adresse IP est privée (0/1)
- `IpFirstOctet` : premier octet de l'adresse IP (0-255)

Variables dérivées (ratios comportementaux)

- $\text{MonetaryPerDay} = \frac{\text{MonetaryTotal}}{\text{Recency} + 1}$
- $\text{AvgBasketValue} = \frac{\text{MonetaryTotal}}{\text{Frequency}}$
- $\text{TenureRatio} = \frac{\text{Recency}}{\text{CustomerTenureDays}}$

Ces indicateurs représentent mieux le comportement client. Les divisions sont protégées contre les valeurs nulles.

Nettoyage initial

- Suppression de `CustomerID` : identifiant sans pouvoir prédictif
- Suppression des colonnes **constantes** (variance nulle, aucune information discriminante)

Ces suppressions sont déterministes (elles ne dépendent d'aucune statistique apprise) : elles peuvent donc être appliquées avant le split sans induire de leakage.

Détection précoce de features suspectes

Le preprocessing inclut également une étape de contrôle des variables « trop prédictives » :

- catégories **pures** (une modalité associée quasi-exclusivement à une seule classe),
- variables numériques avec corrélation très élevée à la cible,
- valeurs numériques mappant de manière quasi-déterministe une classe.

Cette étape est appliquée uniquement quand la cible est disponible (phase training), et exclut certaines variables métier *protégées* pour éviter des suppressions abusives.

1.4 Séparation des données (train / test)

Après les transformations déterministes, les données sont divisées en deux ensembles :

- **Ensemble d'entraînement : 80%**
- **Ensemble de test : 20%**

Une **stratification** sur la variable cible **Churn** est appliquée afin de conserver la distribution des classes dans les deux ensembles, ce qui est essentiel en présence d'un déséquilibre de classes.

Pourquoi séparer avant les étapes suivantes ?

Toutes les opérations suivantes (imputation, normalisation, encodage, corrélation, VIF) nécessitent d'être **appries (fit) exclusivement sur X_train**, puis **appliquées (transform) sur X_test**.

Effectuer ces opérations avant le split reviendrait à laisser fuir de l'information du jeu de test vers le jeu d'entraînement (*data leakage*), ce qui conduirait à une évaluation des performances **trop optimiste et non réaliste**.

1.5 Phase 2 — Transformations avec fit sur train (après le split)

Pipeline sklearn : imputation, encodage, normalisation

Un pipeline a été construit à l'aide de **ColumnTransformer** et **Pipeline** :

- **Variables numériques** : imputation par la *médiane*, puis normalisation avec **StandardScaler**
- **Variables ordinales** : imputation par la valeur la plus fréquente, puis **OrdinalEncoder** avec catégories ordonnées explicites
- **Variables nominales** : imputation par la valeur la plus fréquente, puis **OneHotEncoder**

Encodage ordinal — exemples d'ordre préservé

- Low < Medium < High < VIP
- Faible < Moyen < Élevé < Critique
- Nouveau < Jeune < Établi < Ancien

Le pipeline est **fit** uniquement sur **X_train**, puis **transform** est appliqué sur X_train et X_test séparément.

Résultat après le pipeline sklearn

Nombre initial de features (avant encodage) : ~ 50

Nombre après encodage sklearn : 105 features

- **Numériques normalisées** : 30 features
- **Ordinales encodées** : 15 features (5 variables × 3 catégories moyenne)
- **Nominales one-hot** : 60 features (OneHotEncoder crée une colonne par modalité)

Suppression des features corrélées

Après encodage, une étape de filtrage par corrélation est appliquée sur les features numériques.

Méthode de suppression par corrélation de Pearson

- Calcul de la matrice de corrélation absolue $|r|$ sur **X_train uniquement**
- Extraction de la matrice triangulaire supérieure (sans diagonale)
- Pour chaque paire dont $|r| > 0,90$: suppression de la seconde feature
- Application de la même liste de suppressions sur **X_test**

Seuil par défaut : $|r| = 0,90$ (paramétrable via `-corr-threshold`)

Résultat du filtrage par corrélation

Features supprimées (corrélation > 0.90) : 17 features

- MonetaryMin, MonetaryMax (redondants avec MonetaryTotal)
- MinQuantity, MaxQuantity (dérivés linéairement de Frequency)
- UniqueDescriptions, UniqueInvoices (fortement corrélés)
- Variables géographiques redondantes (Country_Australia, Country_Brazil, etc.)

Features restantes après corrélation : $105 - 17 = 88$ features

Suppression par VIF (Variance Inflation Factor)

En complément, une suppression itérative basée sur le **VIF** élimine la multicolinéarité résiduelle non détectée par la corrélation par paires.

Définition du VIF

$$\text{VIF}_j = \frac{1}{1 - R_j^2}$$

où R_j^2 est le coefficient de détermination de la régression de la feature j sur toutes les autres.

Valeur du VIF	Interprétation
< 5	Pas de multicolinéarité
$5 - 10$	Multicolinéarité modérée
> 10	Multicolinéarité élevée — suppression recommandée

Algorithme de suppression itérative

1. Calculer le VIF de toutes les features numériques sur **X_train**
2. Identifier la feature avec le VIF le plus élevé
3. Si $\text{VIF} > \text{seuil}$: supprimer cette feature et retourner à l'étape 1
4. Sinon : arrêt — appliquer les mêmes suppressions sur **X_test**

Seuil par défaut : $\text{VIF} = 10,0$ (paramétrable via `-vif-threshold`)

Important : Le calcul du VIF est **désactivé par défaut** (flag `-enable-vif`) pour éviter un temps de compilation excessif. Il ne s'active que si l'utilisateur le demande explicitement.

1.6 Phase 3 — Détection et suppression du data leakage

Une étape critique a été ajoutée pour détecter et éliminer les features causant du *data leakage* — c'est-à-dire les variables qui ne seraient pas disponibles au moment de la prédiction en production, ou qui sont des proxies directs du phénomène à prédire.

Définition et problématique

Qu'est-ce que le data leakage ?

Le **data leakage** (fuite de données) se produit quand une feature apporte une information **directe ou indirecte** sur la variable cible, sans refléter une causalité réelle.

Exemples typiques :

- Une feature qui est **la cible déguisée** (ex : **Recency** mesurant les jours depuis le dernier achat, qui est un proxy du churn)
- Une feature **calculée après** le comportement à prédire (ex : utiliser des transactions futures pour prédire le churn passé)
- Une feature **disponible uniquement pour une classe** (ex : un code d'erreur n'apparaissant que chez les clients qui ont quitté)

Conséquence : Le modèle atteint une précision **artificiellement élevée** (souvent proche de 1.0) qui **ne se reproduit pas** en production.

Cas détecté dans ce projet

Au cours du développement, l'entraînement du modèle affichait une **accuracy parfaite de 1.0**, ce qui était hautement suspect. Une investigation a révélé :

Investigation : Pourquoi 1.0 d'accuracy ?

1. Calcul de la corrélation de chaque feature avec la cible sur X_{train}
2. Identification des features avec $|r| > 0.3$ (corrélation forte)
3. **Découverte critique** : **Recency** affichait une corrélation de **0.859** avec Churn

La variable **Recency** mesure le nombre de jours depuis le dernier achat. Elle est **par construction** un excellent proxy du churn : un client inactif depuis longtemps est très probablement churné. Cela crée une relation quasi-déterministe :

$$\text{Churn} \approx \begin{cases} 0 & \text{si Recency petit} \\ 1 & \text{si Recency grand} \end{cases}$$

Le problème : En production, nous voudrions prédire le churn **avant** qu'il n'advienne, pas après en voyant l'inactivité. Utiliser **Recency** revient à « prédire » un phénomène dont on connaît déjà l'outcome via une mesure directe.

Solution : Détection automatique du leakage

Une fonction `remove_leakage_features()` a été implémentée pour identifier et supprimer automatiquement les features problématiques.

Algorithme de détection du leakage

1. Calculer la corrélation de Pearson de chaque variable numérique avec la cible (**Churn**) sur X_{train} uniquement
2. Pour chaque feature : si $|\text{corrélation}| > \text{seuil}$ (défaut 0.3), la marquer pour suppression
3. Appliquer la même liste de suppressions sur X_{test}
4. Logger les features supprimées et leurs corrélations pour transparence

Seuil par défaut : $|r| = 0,3$ (paramétrable via `-leakage-threshold`)

Features supprimées (data leakage detection)

Nombre de features supprimées : 13 features

Parmi lesquelles :

- **Recency** ($r = 0,859$) — Principal responsable de l'accuracy artificielle
- **CustomerTenureDays** ($r = 0,712$) — Proxy indirect du churn
- **PreferredMonth**, **SpendingCategory**, **ChurnRiskCategory** — Dérivées ou proxies
- **RFMSegment_*** (Champions, Dormants, etc.) — Segmentation basée sur comportement passé
- **CustomerType_*** (Occasionnel, Perdu, Régulier, etc.) — Catégorisation corrélée au churn
- **FavoriteSeason_*** (Automne, Printemps) — Préférences comportementales
- **ProductDiversity_Explorateur** — Diversité produits proxy du comportement

Impact sur les performances

Avant suppression du leakage : Accuracy 1.0000 (irréaliste)

Après suppression du leakage : Accuracy 0.9394 (réaliste et défendable)

Interprétation :

- Le modèle préserve **94%** de précision même sans accès à Recency
- Les 6% d'erreurs restantes reflètent la vraie complexité du problème
- Le modèle est maintenant **transposable** en production

2 Entraînement du Modèle (Training)

2.1 Objectif et architecture

Après le preprocessing complet, les données sont prêtes pour l'entraînement d'un modèle de classification binaire capable de prédire le churn client.

Pipeline d'entraînement

#	Étape
1	Charger les données prétraitées (X_train, X_test, y_train, y_test)
2	Construire le preprocessor sklearn (ColumnTransformer)
3	Fit du preprocessor sur X_train et transformation
4	Appliquer les filtres : corrélation, VIF, détection leakage
5	Instancier le modèle (LogisticRegression ou RandomForestClassifier)
6	Entraîner sur les features nettoyées
7	Évaluer sur le jeu de test
8	Sauvegarder le modèle et les métriques

2.2 Choix du modèle

Trois modèles peuvent être sélectionnés via l'argument `-model-type` :

- **LogisticRegression** (défaut) : Modèle linéaire simple, interprétable, rapide
- **RandomForestClassifier** : Modèle ensemble, non-linéaire, plus flexible
- **SVC** : Modèle à marges maximales avec estimation de probabilités

Configuration des modèles

LogisticRegression :

- Poids équilibrés pour les classes (`class_weight='balanced'`)
- Résolveur : `lbfgs`
- Random state : paramétrable (défaut 42)

RandomForestClassifier :

- 300 arbres (`n_estimators=300`)
- Poids équilibrés par sous-échantillonnage (`class_weight='balanced_subsample'`)
- Parallélisation CPU (`n_jobs=-1`)
- Random state : paramétrable (défaut 42)

SVC :

- Pondération des classes (`class_weight='balanced'`)
- Probabilités activées (`probability=True`)

2.3 Rééquilibrage des classes

Avant l'entraînement, le script applique automatiquement une stratégie de rééquilibrage :

- **SMOTE** si `imblearn` est disponible ;
- sinon **oversampling aléatoire** de la classe minoritaire pour égaliser les effectifs.

Ce mécanisme améliore la détection de la classe churn (minoritaire) et limite le biais vers la classe majoritaire.

2.4 Processus d'évaluation

Le modèle est évalué sur l'ensemble de test via plusieurs métriques :

Métrique	Définition / Interprétation
Accuracy	Proportion d'observations correctement classées : $\frac{TP + TN}{Total}$
Precision	Fiabilité des prédictions positives : $\frac{TP}{TP + FP}$
Recall	Couverture des vraies churns détectées : $\frac{TP}{TP + FN}$
F1-Score	Moyenne harmonique de Precision et Recall : $2 \cdot \frac{Precision \times Recall}{Precision + Recall}$
ROC-AUC	Aire sous la courbe Receiver Operating Characteristic (probabilités)
Confusion Matrix	Tableau : $\begin{pmatrix} TN & FP \\ FN & TP \end{pmatrix}$

2.5 Résultats finaux (Logistic Regression)

Métriques d'évaluation du modèle (LogisticRegression)

Métrique	Valeur
Accuracy	0.9394 (93.94%)
Precision	0.8940 (89.40%)
Recall	0.9278 (92.78%)
F1-Score	0.9106 (91.06%)
ROC-AUC	0.9728 (97.28%)

Confusion Matrix :

$$\begin{pmatrix} 552 & 32 \\ 21 & 270 \end{pmatrix}$$

- **True Negatives (TN)** : 552 clients correctement prédits non-churné
- **False Positives (FP)** : 32 clients faux positifs (prédits churné, ne l'étaient pas)
- **False Negatives (FN)** : 21 clients faux négatifs (non-détectés churné)
- **True Positives (TP)** : 270 clients correctement prédits churné

Interprétation :

- Le modèle détecte **92.78%** des churns (recall élevé) → bon pour la rétention
- Parmi les clients qu'il classe comme churné, **89.40%** le sont vraiment (precision élevée) → peu de faux positifs
- ROC-AUC 0.9728 indique une excellente discrimination entre les deux classes

3 Prédiction par lots (Batch Prediction)

3.1 Objectif

Une fois le modèle entraîné et sauvegardé, il peut être utilisé pour faire des prédictions sur de nouvelles données. Le script `predict.py` permet une prédiction par lots (batch inference) sur une

nouvelle liste de clients.

3.2 Architecture et workflow

Pipeline de prédiction

1. Charger le bundle sauvegardé (modèle + preprocessor + métadonnées)
2. Charger les données de test (sans labels)
3. Appliquer les mêmes transformations que le training :
 - Feature engineering (dates, IP, ratios)
 - Encodage et normalisation
 - Réindexation aux features finales (remaplage)
4. Appliquer le modèle entraîné
5. Générer les prédictions et probabilités
6. Sauvegarder les résultats

3.3 Format du bundle sauvegardé

Le modèle est sauvegardé dans un fichier `churn_model_bundle.joblib` contenant un dictionnaire avec :

Clé	Contenu
<code>model</code>	Modèle scikit-learn entraîné (LogisticRegression ou RandomForest)
<code>preprocessor</code>	ColumnTransformer contenant l'état fit du preprocessing
<code>raw_feature_columns</code>	Colonnes brutes attendues en entrée avant preprocessing
<code>reference_row</code>	Ligne client de référence pour l'inférence unitaire Flask
<code>feature_names_out</code>	Noms des features après preprocessing
<code>input_columns</code>	Noms des colonnes d'entrée (avant preprocessing)
<code>final_columns</code>	Noms des colonnes finales (après tous les filtres)
<code>kept_feature_names</code>	Liste explicite des features finales conservées
<code>target_col</code>	Nom de la colonne cible (Churn)
<code>model_type, random_state</code>	Hyperparamètres du modèle
<code>sampling_applied</code>	Méthode de rééquilibrage appliquée (SMOTE / oversampling)
<code>metrics</code>	Dictionnaire des métriques d'évaluation

Avantages du bundle

- **Reproductibilité** : Toutes les transformations sont encapsulées dans un seul fichier
- **Portabilité** : Le bundle peut être déployé sur n'importe quel serveur/environnement
- **Traçabilité** : Les métadonnées permettent de suivre la provenance du modèle
- **Cohérence** : Les données de production sont transformées exactement comme pendant le training

3.4 Prédictions produites

Le script `predict.py` génère un fichier CSV avec les colonnes :

- **prediction** : Prédiction binaire (0 = non-churné, 1 = churné)
- **probability** : Probabilité d'être churné (entre 0 et 1)

Exemple de sortie de prédiction

Row	prediction	probability
0	0	0.1234
1	1	0.8956
2	0	0.3421
⋮	⋮	⋮

4 Application Web Flask (Démon Interactive)

4.1 Objectif

En complément de la prédiction batch, une application web Flask a été développée pour permettre une démonstration interactive de la prédiction du churn sur un client unique.

4.2 Architecture de l'application

Le fichier `app/app.py` implémente :

- Le chargement du bundle entraîné (`models/churn_model_bundle.joblib`)
- Le chargement des métriques (`models/training_metrics.json`)
- Une interface web (template HTML) pour saisir des variables client
- Une API JSON pour retourner la prédiction et le niveau de risque
- Des fallbacks d'affichage si les métriques ne sont pas disponibles au démarrage

Routes principales de l'application

- GET / : affiche l'interface principale et les métriques du modèle
- POST /predict : reçoit les entrées utilisateur et retourne la prédiction churn
- GET /health : endpoint technique de vérification d'état (*health check*)

4.3 Workflow de prédiction unitaire

Lorsqu'un utilisateur soumet un formulaire :

1. L'application construit une ligne client à partir d'une `reference_row` sauvegardée dans le bundle
2. Les champs saisis (`Recency`, `Frequency`, `MonetaryTotal`, `CustomerTenureDays`, `SatisfactionScore`, `SupportTicketsCount`) remplacent les valeurs par défaut
3. Les mêmes transformations que le training sont appliquées (dates, IP, ratios, preprocessor sklearn)
4. Les features sont réalignées strictement sur les colonnes finales du modèle
5. Le modèle retourne :
 - la classe prédite (`churn = 0/1`)
 - la probabilité (`probability`)

4.4 Niveaux de risque métier

La probabilité prédite est transformée en niveau de risque lisible pour l'utilisateur :

- **Faible** si $p < 0.4$
- **Modéré** si $0.4 \leq p < 0.7$
- **Élevé** si $p \geq 0.7$

Exemple de réponse JSON de /predict

- `churn` : 0 ou 1
- `probability` : score entre 0 et 1
- `probability_pct` : score formaté en pourcentage
- `risk_level` : Faible / Modéré / Élevé
- `label` : « Client fidèle » ou « Client à risque »

4.5 Exécution de la démo

- Lancer : `python app/app.py`
- Ouvrir : `http://localhost:5000`
- Tester différents profils clients pour observer l'évolution du risque churn

5 Résumé statistique du pipeline complet

Réduction progressive des features

Étape	Nombre de features	Suppressions
Features initiales (brutes)	50	—
Après feature engineering	53	+3 (dates, IP, ratios)
Après encoding sklearn	105	+52 (OneHotEncoder)
Après suppression corrélation	88	−17
Après suppression VIF	88	−0 (désactivé par défaut)
Après détection leakage	75	−13

Réduction totale : De 105 features post-encoding à 75 features finales = **28.6% de réduction**, tout en préservant 93.94% de précision.

6 Conclusion

Ce projet a démontré l'importance cruciale d'une rigueur méthodologique en Machine Learning :

1. **Respect de l'ordre du pipeline** : Chaque étape a été placée avant ou après le split train/test selon les principes de prévention du data leakage
2. **Détection du leakage** : L'identification de **Recency** comme proxy du churn a révélé l'existence d'une fuite critique qui aurait conduit à des performances en production totalement irréalistes

3. **Métriques réalistes** : Après suppression des 13 features problématiques, le modèle affiche des performances réalistes (93.94% accuracy, 97.28% ROC-AUC) qui reflètent la vraie complexité du problème
4. **Qualité de la sélection des features** : La réduction de 105 à 75 features (28.6% de réduction) élimine la redondance et améliore l'interprétabilité sans sacrifier les performances
5. **Reproductibilité et production-readiness** : L'utilisation d'un bundle joblib garantit que le modèle déployé en production appliquera exactement les mêmes transformations qu'en training

Points clés pour la production

- Le modèle est prêt pour le déploiement avec des performances défendables
- Les 75 features finales ne contiennent aucune fuite d'information
- Le recall de 92.78% est excellent pour la rétention client
- L'architecture modulaire permet un retraining facile sur de nouvelles données